P.Vishwa Nandan(192473015)

Q4 – Choosing a Map for a Pharmacy Inventory

Marks: 6

This question is about comparing three Java Map collections:

1. HashMap
2. TreeMap
3. LinkedHashMap

Problem Context – Hospital Pharmacy Stock Lookup

Imagine a hospital pharmacy that stores medicines using unique medicine codes.

Pharmacists need to:

- Search medicine stock using its medicine code.
- Print all medicines in ascending medicine-code order.
- Print medicines in the order they were added.

We must determine which Map is most suitable for these operations. Pasted text(6)

---

1. Understanding HashMap, TreeMap and LinkedHashMap

A. HashMap

HashMap stores information as key-value pairs.

Example:

```
HashMap<String, Integer> stock = new HashMap<>();


stock.put("MED103", 50);

stock.put("MED101", 30);

stock.put("MED102", 40);


System.out.println(stock.get("MED101"));
```

Output:

30

Here:

- Key = Medicine code.
- Value = Available stock.

Advantages:

- Fast searching.
- Fast insertion.
- Does not maintain any guaranteed order.

Disadvantage:

- Cannot directly produce sorted or insertion-order reports.

B. TreeMap

TreeMap stores key-value pairs in sorted key order.

Example:

```
TreeMap<String, Integer> stock = new TreeMap<>();


stock.put("MED103", 50);

stock.put("MED101", 30);

stock.put("MED102", 40);


System.out.println(stock);
```

Output:

{MED101=30, MED102=40, MED103=50}

Advantages:

- Automatically sorts keys.
- Produces medicine codes in ascending order.
- Supports efficient searching.

Disadvantage:

- Searching and insertion take O(log n) time.
- Does not preserve insertion order.

C. LinkedHashMap

LinkedHashMap maintains the order in which elements were inserted.

Example:

```
LinkedHashMap<String, Integer> stock = new LinkedHashMap<>();


stock.put("MED103", 50);
stock.put("MED101", 30);
stock.put("MED102", 40);


System.out.println(stock);
```

Output:

{MED103=50, MED101=30, MED102=40}

Advantages:

- Maintains insertion order.
- Provides average O(1) searching.
- Useful for displaying medicines in the order they were added.

Disadvantage:

- Does not automatically sort medicine codes.

---

2. Prediction Before Implementation

Before executing the program, we can predict the behaviour of each Map.

Time Complexity Comparison

Operation	HashMap	TreeMap	LinkedHashMap
put()	O(1) average	O(log n)	O(1) average
get()	O(1) average	O(log n)	O(1) average
Iteration	O(n)	O(n)	O(n)
Code-order report	O(n log n)	O(n)	O(n log n)
Insertion-order report	Requires extra list	Requires extra list	O(n)

Here, n represents the number of medicines.

Which Map Produces Each Report?

Medicine code order:

TreeMap naturally produces medicines in ascending medicine-code order.

HashMap and LinkedHashMap require sorting their keys before generating the report.

Insertion order:

LinkedHashMap naturally preserves insertion order.

HashMap and TreeMap require an additional list to remember the original insertion order.

---

3. Design the Experiments

We will test three different inventory sizes.

Experiment 1: 1,000 medicines.

Experiment 2: 10,000 medicines.

Experiment 3: 50,000 medicines.

For each inventory size, we measure:

1. Time required for 100,000 medicine lookups.
2. Time required to generate the medicine-code report.
3. Time required to generate the insertion-order report.
4. Whether the report follows the required order.

This helps us identify the most suitable Map for the pharmacy.

5. Understanding the Program

There are three important methods in this program.

A. createInventory()

map.put(code, medicine);

This inserts each medicine into the selected Map.

The same medicine data is inserted into HashMap, TreeMap and LinkedHashMap.

We deliberately insert medicine codes in descending order:

MED003

MED002

MED001

This helps us verify the difference between sorted order and insertion order.

B. buildCodeReport()

This method generates the medicine report in ascending code order.

For TreeMap, we directly iterate through its values because it already sorts its keys.

For HashMap and LinkedHashMap, we first sort the medicine codes using:

Collections.sort(keys);

Then we retrieve the corresponding medicines.

C. buildInsertionReport()

This method generates the report in the original insertion order.

LinkedHashMap already maintains insertion order.

HashMap and TreeMap use an additional insertionOrder list to remember when medicines were added.

D. averageLookupTime()

This method performs 100,000 lookups and averages the execution time over five runs after warm-up.

We use:

System.nanoTime();

to measure execution time.

The program also displays the average time taken by one lookup.

Measurement note: These are local in-memory microbenchmarks. Actual measurements depend on your laptop, Java version, runtime optimisation and system load. They do not include network or database latency.

---

6. Expected Output – Order Verification

The first part of the program should display:

ORDER VERIFICATION

HashMap

Code order correct: true

Insertion order correct: true

TreeMap

Code order correct: true

Insertion order correct: true

LinkedHashMap

Code order correct: true

Insertion order correct: true

Why does this work?

We explicitly generate the required ordering when necessary.

For example:

- HashMap uses sorting for code order and an additional list for insertion order.
- TreeMap uses its natural sorted order and an additional list for insertion order.
- LinkedHashMap uses its insertion order and sorting for code order.

Important: HashMap itself does not guarantee insertion or sorted order. The ordering is produced by the additional logic.

---

7. Experimental Results

The program performs the following experiments.

Experiment 1 – 1,000 Medicines

Measurement	HashMap	TreeMap	LinkedHashMap
100,000 lookups	Actual output	Actual output	Actual output
Code-order report	Actual output	Actual output	Actual output
Insertion-order report	Actual output	Actual output	Actual output

Experiment 2 – 10,000 Medicines

Measurement	HashMap	TreeMap	LinkedHashMap
100,000 lookups	Actual output	Actual output	Actual output
Code-order report	Actual output	Actual output	Actual output
Insertion-order report	Actual output	Actual output	Actual output

Experiment 3 – 50,000 Medicines

Measurement	HashMap	TreeMap	LinkedHashMap
100,000 lookups	Actual output	Actual output	Actual output
Code-order report	Actual output	Actual output	Actual output
Insertion-order report	Actual output	Actual output	Actual output

Note: Enter the actual times printed by your program. Do not invent benchmark values.

---

8. Scalability Inquiry and Deployment Decision

The pharmacy performs hundreds of medicine lookups every day.

For this system, we define the acceptable response time for one in-memory lookup as **less than or equal to 1 millisecond**.

The program measures 100,000 lookups and calculates the average time per lookup.

For example, the calculation is:

Average lookup time = Total lookup time / 100,000.

After executing the program, compare the measured results with the 1 ms requirement.

Recommended Map: LinkedHashMap

I recommend LinkedHashMap for the pharmacy inventory because:

1. It provides average O(1) lookup.
2. It maintains medicine insertion order automatically.
3. It can generate the insertion-order report without an additional list.
4. Code-order reports can be generated by sorting the medicine codes.
5. It supports all the required operations.

However, sorting keys for a code-order report requires O(n log n) time.

When would TreeMap be better?

If the pharmacy frequently generates sorted medicine reports and wants sorted keys maintained automatically, TreeMap is a suitable choice.

It provides O(log n) lookup and O(n) iteration in sorted order.

Why Is Hashtable Not Preferred in New Code?

Hashtable is an older Java collection.

Its methods are synchronized, which introduces synchronization overhead for individual operations.

It also does not allow null keys or null values.

For modern Java applications:

- Use HashMap for ordinary non-thread-safe map operations.
- Use LinkedHashMap when insertion order is required.
- Use TreeMap when sorted keys are required.
- Use ConcurrentHashMap when concurrent access is needed.

Therefore, Hashtable is generally not the preferred choice for new Java code.

---

9. Alternative Concept – Sorted Array and Arrays.binarySearch()

Instead of using Maps, the pharmacy can store medicines in an array sorted by medicine code.

Example:

```
import java.util.Arrays;

import java.util.Comparator;


class Medicine {


   String code;

   String name;

   int stock;


   Medicine(String code, String name, int stock) {

      this.code = code;

      this.name = name;

      this.stock = stock;

   }

}
```

```
public class Main {

    public static void main(String[] args) {

        Medicine[] medicines = {

            new Medicine("MED001", "Paracetamol", 50),

            new Medicine("MED002", "Amoxicillin", 40),

            new Medicine("MED003", "Ibuprofen", 30)
        };

        // Search for a medicine
        int index = Arrays.binarySearch(
                medicines,
                new Medicine("MED002", "", 0),
                Comparator.comparing(m -> m.code)
        );

        if (index >= 0) {

            System.out.println(
                    medicines[index].code + " - " +
                    medicines[index].name
            );
        }
    }
}
```

Output:

MED002 - Amoxicillin

How It Works

The array is maintained in ascending order by medicine code.

Arrays.binarySearch() repeatedly checks the middle element and reduces the search range.

This makes searching faster than a normal linear search.

Complexity Analysis

Operation	Sorted Array	HashMap	TreeMap
Search	O(log n)	O(1) average	O(log n)
Insertion	O(n)	O(1) average	O(log n)
Code-order report	O(n)	O(n log n) after sorting	O(n)
Space	O(n)	O(n)	O(n)

Sorted Array:

- Searching takes O(log n).
- Insertion takes O(n) because elements must be shifted.
- Code-order reporting takes O(n) because the array is already sorted.
- Space complexity is O(n).

HashMap:

- Searching takes O(1) average.
- Insertion takes O(1) average.
- Sorting is required for code-order reporting.
- Space complexity is O(n).

TreeMap:

- Searching takes O(log n).
- Insertion takes O(log n).
- Code-order reporting takes O(n).
- Space complexity is O(n).

Comparison

A sorted array provides efficient searching and reporting but makes insertion expensive.

HashMap provides faster average searching but does not maintain sorted order.

TreeMap maintains sorted order automatically and supports efficient insertion and searching.

Therefore, Maps are generally more flexible for a pharmacy inventory that frequently adds medicines and searches their stock.

---

10. Conclusion

HashMap, TreeMap and LinkedHashMap provide different advantages for pharmacy inventory management. HashMap is suitable for fast average-case lookups, TreeMap automatically maintains sorted medicine codes, and LinkedHashMap preserves insertion order. Based on the requirement for fast lookups and insertion-order reporting, LinkedHashMap is recommended. Code-order reporting can be produced by sorting its keys when needed.